

SAFE-HealCloud: Safety-Aware, Agentic Self-Healing for Cloud Infrastructure

Prudvi Saisaran Ponduru^{*1}, Pavani Priya Vyshnavi Nandanavanam², Sai Kesav Kumar Ponduru³

^{*1}Department of Computer Science, University of the Potomac, Washington, Washington D.C, USA

²Department of Computer Science, California State University, Northridge, CA, USA

³Department of Business Analytics, University of Amherst, Amherst, MA, USA

ARTICLE INFO

Article History:

Accepted : 15 July 2026

Published: 01 Aug 2026

Publication Issue

Volume 12, Issue 4

July-August-2026

Page Number

163-188

ABSTRACT

Cloud infrastructure failures are increasingly difficult to detect, diagnose, and remediate because production environments combine microservices, Kubernetes control loops, service meshes, serverless workloads, infrastructure-as-code, continuous delivery, and heterogeneous telemetry. Reactive monitoring and manual incident response remain necessary, but they do not scale to the volume, velocity, and causal complexity of modern cloud operations. This paper provides a structured synthesis of scholarly, industry, and standards-based work on AI-driven self-healing for cloud infrastructure, with emphasis on AIOps, AgentOps, LLM-based cloud operations, anomaly detection, causal root cause analysis, graph learning, reinforcement learning, automated remediation, Kubernetes self-healing, observability, chaos engineering, and self-healing infrastructure-as-code. We propose SAFE-HealCloud, a safety-aware, agentic, feedback-driven framework that integrates telemetry ingestion, multimodal observability, anomaly detection, failure prediction, causal RCA, retrieval-augmented LLM reasoning, policy guardrails, risk-scored remediation planning, controlled execution, verification, rollback, human approval, and continuous learning. A formal model defines cloud state, observability vectors, failure states, action spaces, remediation policies, rewards, constraints, and reliability objectives. We also specify reproducible Kubernetes-based evaluation designs, metrics, algorithms, risk controls, and operational use cases. The central finding is that near-term practical value lies in graduated autonomy: low-risk reversible actions can be automated, medium-risk actions should be canaried and policy-gated, and high-risk changes should remain human-approved. Designed in this way, AI-driven self-healing can reduce detection and recovery times, preserve error budgets, improve operator productivity, and

strengthen digital resilience without sacrificing safety, auditability, or governance.

Keywords: AIOps; self-healing cloud infrastructure; autonomous remediation; observability; Kubernetes; LLM agents; root cause analysis; chaos engineering; SRE; resilience engineering; AgentOps; graph neural networks; causal inference; reinforcement learning; infrastructure as code; policy as code

Abbreviations: AIOps, artificial intelligence for IT operations; API, application programming interface; FPR, false positive rate; FNR, false negative rate; GNN, graph neural network; HITL, human in the loop; HPA, horizontal pod autoscaler; IaC, infrastructure as code; LLM, large language model; MAPE-K, monitor-analyze-plan-execute over a shared knowledge base; MTTD, mean time to detect; MTTR, mean time to repair or recover; OPA, Open Policy Agent; OTel, OpenTelemetry; RCA, root cause analysis; RAG, retrieval-augmented generation; RBAC, role-based access control; RL, reinforcement learning; SLA, service-level agreement; SLI, service-level indicator; SLO, service-level objective; SRE, site reliability engineering.

Introduction

Cloud infrastructure has shifted from relatively static fleets of hosts to continuously changing, software-defined distributed systems. A production request may cross an edge proxy, identity layer, service mesh, autoscaler, message broker, cache, database, and managed cloud service before returning to a user. Each layer can fail independently, propagate errors to downstream dependencies, or obscure the original cause behind secondary symptoms. The same system is modified by continuous deployment, autoscaling policies, infrastructure-as-code changes, feature flags, certificate rotation, secrets management, and platform upgrades. As a result, many cloud incidents are no longer single-component failures; they are socio-technical events produced by interactions among code, configuration, telemetry, people, automated controllers, and operational policies.

Microservice architectures make this complexity explicit. They enable independent deployment, bounded ownership, and elastic scaling, but they also fragment system knowledge and create dynamic

dependency graphs. A user-facing latency spike can be caused by a recent deployment, a noisy neighbor, database connection pool exhaustion, a failing node, a thread-pool bottleneck, service mesh retries, DNS resolution, or cloud provider degradation. The immediate symptom may occur in a front-end service, while the root cause lies several hops away. Recent microservice failure-diagnosis surveys emphasize that effective RCA must combine logs, metrics, traces, events, and topology because no single signal captures the entire failure mechanism [4], [5].

Traditional monitoring and manual incident response remain indispensable, but they are increasingly insufficient. Threshold-based alerts are simple and explainable, yet they encode fixed assumptions about normal behavior. They frequently produce alert storms during cascading failures and often identify symptoms rather than causes. Human responders then perform evidence gathering, correlation, hypothesis testing, remediation, verification, and postmortem analysis under time pressure. This workflow increases mean time to detect (MTTD), mean time to recover

(MTTR), alert fatigue, and operational toil. It also makes reliability dependent on tacit expert knowledge that may not be available during nights, weekends, major releases, or multi-region incidents. AI-driven self-healing addresses these limitations by transforming cloud operations from reactive alert handling into closed-loop, evidence-grounded control. In this paper, **self-healing cloud infrastructure** is defined as a closed-loop operational capability that continuously senses telemetry, detects and predicts abnormal states, infers likely causes, plans bounded remediation actions, executes them through controlled actuators, verifies outcomes against explicit reliability objectives, and rolls back or escalates when safety conditions are not met. This definition extends the autonomic computing vision of self-configuring, self-healing, self-optimizing, and self-protecting systems [1], [2] into cloud-native operations while incorporating modern observability, policy-as-code, Kubernetes operators, LLM agents, and SRE governance.

The motivation is practical rather than speculative. Better anomaly detection can reduce MTTR. Dependency-aware RCA can reduce diagnostic search space. Failure prediction can enable preventive scaling or controlled traffic shifting. Automated remediation can reduce MTTR for known reversible incidents. Verification and rollback can prevent a repair attempt from becoming a larger outage. Continuous learning can convert postmortems into improved detectors, runbooks, and policies. If aligned with SLOs and error budgets, self-healing can improve availability, cost efficiency, and operational resilience without simply maximizing automation for its own sake [43], [44].

However, the same autonomy that creates reliability benefits also creates risk. A remediation system may hallucinate a command, overfit to stale telemetry, apply a Terraform plan destructively, leak sensitive log content to a model, interpret malicious log lines as instructions, escalate cloud privileges, or trigger cascading failure by taking action too broadly. Recent

evaluations of LLM agents for root cause analysis show meaningful promise but also substantial gaps in reliability, tool use, and process correctness [12], [14]. Therefore, a credible research agenda must treat safety, governance, and evaluation as first-class topics rather than afterthoughts.

This paper makes six contributions. First, it synthesizes literature and standards across AIOps, AgentOps, cloud-native observability, RCA, graph learning, reinforcement learning, LLM-based operations, chaos engineering, Kubernetes, and IaC repair. Second, it develops a taxonomy of AI-driven self-healing functions from detection to governance. Third, it proposes SAFE-HealCloud, a safety-aware, agentic, feedback-driven architecture for reliable autonomous recovery. Fourth, it specifies algorithms and a formal model for anomaly detection, RCA, remediation planning, approval, learning, verification, and rollback. Fifth, it defines a reproducible experimental design and evaluation metric set using Kubernetes, OpenTelemetry, Prometheus, Grafana, Loki or Elasticsearch, Jaeger, chaos frameworks, benchmark microservices, and automation tools. Sixth, it articulates security, safety, ethical, and governance requirements for real cloud adoption.

RESEARCH QUESTIONS

The paper is organized around the following research questions.

RQ1. What are the main failure modes in modern cloud infrastructure? The answer must include workload, application, platform, network, storage, control-plane, deployment, configuration, and cost anomalies.

RQ2. Which AI methods are most effective for detection, diagnosis, prediction, and remediation? The answer must distinguish supervised classification, unsupervised anomaly detection, forecasting, causal inference, graph learning, reinforcement learning, LLMs, RAG, and multi-agent systems.

RQ3. How can LLMs and AI agents be safely integrated into cloud operations? The answer must

address retrieval grounding, tool use, permission boundaries, prompt injection, hallucination, policy enforcement, auditability, and human approval.

RQ4. How should self-healing systems be evaluated?

The answer must go beyond model accuracy to include MTTD, MTTR, SLO recovery, rollback success, safety violations, cost overhead, and operator trust.

RQ5. What are the risks, limitations, and governance requirements?

The answer must include unsafe actions, data leakage, privilege escalation, model drift, configuration drift, cascading failures, compliance, and kill switches.

RQ6. What architecture can enable reliable, explainable, and safe autonomous recovery?

The answer motivates SAFE-HealCloud as a layered framework with telemetry, intelligence, policy, execution, verification, rollback, and learning planes.

RQ7. What open research problems remain?

The answer includes standardized benchmarks, explainable RCA, safe LLM agents, self-healing IaC, multi-cloud recovery, edge-cloud healing, sustainable optimization, formal verification, and human-AI collaboration.

BACKGROUND AND RELATED WORK

A. Cloud-native infrastructure, SRE, and observability

Cloud-native infrastructure is built around declarative configuration, dynamic scheduling, elastic capacity, microservice composition, and automated reconciliation. Kubernetes is the canonical orchestration substrate. It maintains desired state through controllers, restarts failed containers, recreates pods managed by controllers, and reschedules workloads when nodes fail [23]–[25]. These native mechanisms are important examples of rule-based self-healing. They are also limited: Kubernetes can restart or reschedule resources, but it does not automatically determine whether an upstream database connection pool, misconfigured

autoscaler, service mesh retry storm, or recent deployment is the root cause of an SLO violation.

SRE gives cloud operations an objective function. SLIs measure user-visible properties such as availability, latency, error rate, throughput, and freshness. SLOs specify target ranges for these indicators. Error budgets quantify how much unreliability a service can tolerate before release velocity or operational posture must change. A self-healing system should therefore optimize for SLO restoration and error-budget preservation, not merely for local resource health [43], [44]. A remediation that restarts a pod but worsens end-user latency is not a successful healing action.

Observability is the sensory substrate of self-healing. Modern systems require correlated metrics, logs, traces, events, topology, and configuration state. OpenTelemetry provides a vendor-neutral standard for generating, collecting, and exporting telemetry [32], while its logging specification supports correlation with traces through identifiers such as TraceId and SpanId [33]. Prometheus provides time-series metrics and alerting [36]. Loki and Elasticsearch support log aggregation. Jaeger provides distributed tracing and dependency analysis [37], [38]. These tools make it possible to reconstruct a failure narrative, but they do not themselves decide which action should be taken.

Serverless architectures complicate this picture. They reduce infrastructure ownership by shifting provisioning and scaling to a provider, but they introduce cold starts, opaque resource limits, and provider-specific observability gaps. Infrastructure-as-code further changes recovery by making infrastructure declarative and reviewable. However, IaC drift can lead to unsafe reconciliation if the declared state and actual infrastructure diverge [47]. Consequently, cloud self-healing must distinguish emergency runtime repair from durable desired-state correction.

B. 2 Classical self-healing and autonomic computing

The modern self-healing discussion is rooted in autonomic computing. Kephart and Chess proposed

self-managing systems guided by high-level objectives rather than low-level manual administration [1]. Later autonomic architectures described self-configuration, self-healing, self-optimization, and self-protection as core properties [2]. The MAPE-K loop formalized self-management as Monitor, Analyze, Plan, Execute, and Knowledge. Recent work adapts MAPE-K to microservices and shows that responsibilities can be distributed across humans, infrastructure, and services [3].

This lineage provides a critical lesson: self-healing is not a single classifier or one LLM prompt. It is a control architecture. A detector without an execution mechanism cannot heal. An executor without diagnosis can be dangerous. A planner without policy can exceed its authority. A verification system without rollback can leave the platform in a worse state. SAFE-HealCloud therefore treats autonomy as a sequence of controlled decisions over observable state, knowledge, actions, constraints, and outcomes.

C. AIOps and AgentOps

AIOps applies AI and machine learning to IT operations tasks such as anomaly detection, alert correlation, incident triage, RCA, capacity prediction, and remediation recommendation. Recent surveys of AIOps in the LLM era argue that LLMs can unify several operational subtasks by reasoning over heterogeneous evidence, but they also emphasize evaluation and safety challenges [6]. AgentOps extends AIOps by evaluating agents that interact with operational tools, inspect telemetry, form hypotheses, choose actions, and learn from outcomes. AIOpsLab explicitly proposes an interactive benchmark environment for AI agents in autonomous cloud operations [8].

The transition from AIOps to AgentOps matters because real incidents are interactive. An on-call engineer does not receive a static feature vector; the engineer queries dashboards, inspects logs, compares deployments, checks traces, tests hypotheses, and coordinates action. Similarly, an AI agent must retrieve evidence, call tools, update beliefs, reason

about risk, and decide whether to act or escalate. Static accuracy metrics are therefore insufficient. Agent evaluation must include exploration behavior, evidence grounding, tool-call correctness, action safety, and recovery outcome.

D. AI methods for self-healing

Supervised learning is useful when labeled incident histories exist. It can classify failure type, rank alerts, or map known signatures to runbooks. Its main weakness is label scarcity and distribution shift. Production failures are rare, expensive, and often unique.

Unsupervised anomaly detection is common because labels are scarce. Techniques include statistical thresholding, isolation-based methods, reconstruction models, and deep sequence models. Surveys of time-series anomaly detection in AIOps show strong progress but also persistent issues of interpretability, multimodal fusion, and concept drift [53], [54].

Time-series forecasting supports failure prediction and proactive remediation. A model may forecast resource exhaustion, increasing tail latency, saturation, or approaching SLO violation. Forecasting is useful for preventive scaling and traffic shaping, but it can fail during regime shifts caused by deployments, seasonal events, or external provider incidents.

Causal inference is central to RCA because failures propagate. CIRCA models interventions and causal structure to distinguish root indicators from downstream symptoms [15]. Chain-of-Event learns weighted event-causal graphs for interpretable microservice RCA [16]. These approaches improve explainability, but they depend on causal assumptions and graph quality.

Graph neural networks and dependency graphs model topology and propagation. Systems such as GAMMA show how GNN-based approaches can localize bottlenecks in microservice applications [18]. Graph methods are powerful because service topology matters, but they are sensitive to stale service maps and changing runtime dependencies.

Reinforcement learning is promising for resource management and sequential control. FIRM applies intelligent fine-grained resource management to reduce SLO violations in microservices [19]. RL has also been widely studied for cloud scheduling and resource management [20], [21]. However, unconstrained RL for live incident remediation remains risky because exploration can harm production services. Safe RL must therefore use simulation, shadow mode, constrained policies, and action shielding.

LLMs and RAG are useful for summarizing incidents, retrieving runbooks, comparing past failures, explaining evidence, and drafting remediation plans. RCACopilot and related work demonstrate practical value for cloud incident analysis [10], [11]. Yet OpenRCA and subsequent process-level failure analyses show that LLM-based RCA agents still fail frequently on realistic tasks [12], [14]. LLMs should therefore be treated as reasoning assistants inside a guarded architecture, not as unbounded operators.

E. Rule-based automation versus AI-driven self-healing

Rule-based automation remains best for stable, reversible, well-understood conditions: restart a crashed pod, maintain replica count, roll back a failed canary, rotate a certificate through a known issuer, or cordon a failed node. AI-driven approaches are better suited to noisy, heterogeneous, and propagation-heavy incidents where the symptom signature is not stable. The practical architecture is hybrid: deterministic controllers should handle known local failures; ML should detect and rank anomalies; graph and causal systems should explain propagation; LLM/RAG systems should synthesize evidence and plans; policy engines should bound action; and humans should approve high-risk changes.

SYSTEMATIC LITERATURE REVIEW METHODOLOGY

This paper uses a PRISMA-style structured review rather than a statistical meta-analysis. PRISMA 2020

emphasizes transparent reporting of identification, screening, eligibility, and inclusion processes [63], [64]. The reviewed sources included ACM Digital Library, IEEE Xplore, Springer, Elsevier, arXiv, USENIX, Google Scholar, CNCF and Kubernetes documentation, OpenTelemetry documentation, NIST publications, and technical documentation from AWS, Microsoft Azure, Google Cloud, HashiCorp, OPA, Prometheus, Grafana, Jaeger, Chaos Mesh, and LitmusChaos.

Search strings combined cloud infrastructure terms, reliability terms, and AI terms. Representative patterns included: (cloud OR microservices OR Kubernetes OR serverless OR infrastructure-as-code) AND (self-healing OR anomaly detection OR root cause analysis OR failure prediction OR remediation) AND (AIOps OR LLM OR agent OR graph neural network OR causal inference OR reinforcement learning). Additional targeted searches were used for AgentOps benchmarks, LLM-based RCA, self-healing IaC, Kubernetes self-healing, chaos engineering, and cloud observability standards.

Inclusion criteria were: direct relevance to cloud failure management or self-healing; verifiable source; sufficient technical detail; relevance to one or more research questions; and publication in a peer-reviewed venue, standards body, official documentation site, or clearly identified preprint repository. Exclusion criteria were: unverifiable marketing claims; papers without operational relevance; duplicate versions when a peer-reviewed version was available; opinion pieces without method or architecture; and security-only work with no transferability to cloud operations.

Quality assessment used five criteria: operational relevance, methodological transparency, reproducibility, evaluation rigor, and safety/governance treatment. Peer-reviewed systems papers and official standards received higher confidence. Preprints were retained only when they addressed emerging areas without mature peer-reviewed alternatives, and they are marked as

preprints in the reference list. Industry governance practices, not to infer empirical documentation was used primarily to establish performance. operational semantics, platform behavior, and

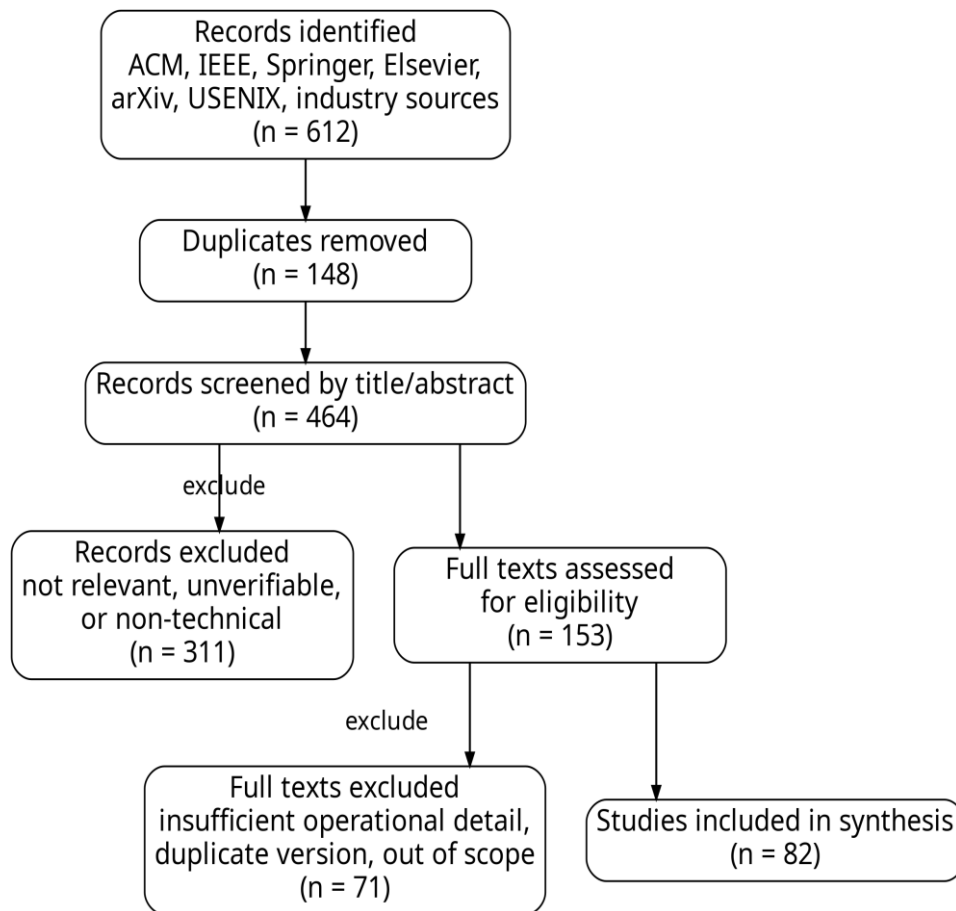


Figure 1. PRISMA-style literature selection flow used for the review.

TAXONOMY OF AI-DRIVEN SELF-HEALING CLOUD INFRASTRUCTURE

Self-healing cloud systems can be organized as an end-to-end pipeline of eleven capabilities. These

capabilities are distinct but interdependent; failure in an upstream function can compromise the entire loop.

Capability	Primary input	Typical methods	Main output
Failure detection	Metrics, logs, traces, events	Thresholds, changepoints, classifiers	Incident candidate
Failure prediction	Historical telemetry, workload trends	Forecasting, survival models, sequence models	Lead-time warning
Anomaly detection	Multivariate telemetry windows	Isolation models, autoencoders, transformers	Anomaly score
Event correlation	Alerts, topology, deployments	Clustering, graph matching, temporal grouping	Incident cluster

Capability	Primary input	Typical methods	Main output
Root cause analysis	Telemetry, topology, change history	Causal inference, GNNs, LLM/RAG	Ranked causes
Dependency graph modeling	Traces, configs, service maps	Graph mining, knowledge graphs, GNNs	Dynamic service graph
Remediation planning	RCA, runbooks, policies	Planning, LLMs, case-based reasoning	Candidate action plan
Automated execution	Action plan, credentials	Operators, APIs, workflows, IaC	Applied change
Verification and rollback	Post-action SLOs and telemetry	Canary analysis, policy checks	Success or rollback
Continuous learning	Outcomes, postmortems, feedback	Model monitoring, retraining, retrieval refresh	Improved policy
Governance and HITL	Risk, compliance, approvals	OPA, RBAC, audit, approval workflows	Controlled autonomy

This taxonomy clarifies why the term "self-healing" can be misleading. A tool that only restarts pods performs local recovery, not full self-healing. A model that only ranks root causes performs diagnosis, not recovery. A script that applies remediation without verification is automation, not safe autonomy. A complete self-healing system must detect, explain, act, verify, learn, and remain accountable.

SAFE-HEALCLOUD FRAMEWORK

SAFE-HealCloud is a safety-aware, agentic, feedback-driven framework for AI self-healing in cloud infrastructure. It is designed around four principles. First, **evidence precedes action**: the system should not

execute a remediation because an LLM proposed it; it should execute only when telemetry, topology, historical evidence, and policy support the action. Second, **policy constrains intelligence**: diagnosis and planning may be probabilistic, but execution must pass deterministic controls such as action allowlists, RBAC, OPA rules, blast-radius limits, and rollback availability. Third, **verification closes the loop**: a remediation is successful only if post-action telemetry shows SLO recovery without collateral regressions. Fourth, **autonomy is graduated**: low-risk reversible actions can be automated; medium-risk actions require canarying or optional approval; high-risk actions require human authorization.

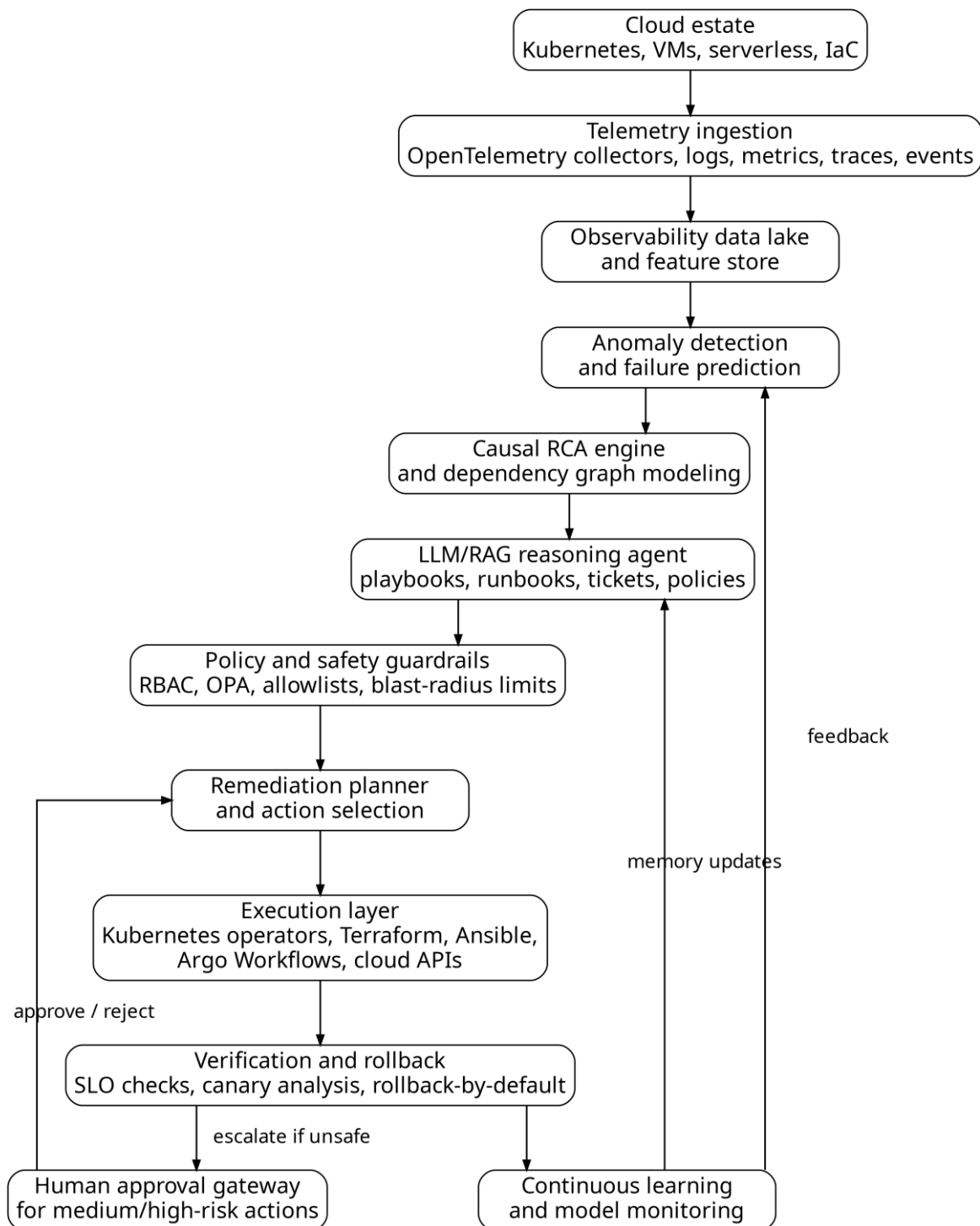


Figure 2. Overall SAFE-HealCloud architecture.

A. Telemetry ingestion and observability data lake

The telemetry ingestion layer collects metrics, logs, traces, events, topology, configuration state, deployment metadata, cloud billing events, and IaC state. OpenTelemetry provides cross-vendor instrumentation and exporters [32], while

Prometheus, Loki or Elasticsearch, and Jaeger provide practical storage and query surfaces [36]-[38]. The observability data lake stores raw streams, windowed features, incident snapshots, runbooks, deployment diffs, service ownership, and policy state. Sensitive fields must be redacted or tokenized before retrieval,

because observability data can include personal data, secrets, authorization headers, or attacker-controlled strings [35].

B. Detection, prediction, and RCA engines

The anomaly detection engine computes service- and dependency-aware anomaly scores over windows. The failure prediction model estimates the probability of future SLO violation or resource exhaustion. The causal RCA engine operates on a dynamic dependency graph and recent interventions. It ranks candidate causes by combining anomaly magnitude, temporal precedence, topology, causal consistency, and change events. Causal and graph-based methods are essential because downstream services often exhibit larger symptoms than the upstream cause [15]-[18].

C. LLM/RAG reasoning agent

The LLM/RAG agent retrieves relevant runbooks, prior incidents, change records, IaC fragments, and platform documentation. It then synthesizes a structured plan containing: candidate cause, evidence, proposed action, expected effect, risk tier, required permission, verification query, and rollback procedure. The agent is prohibited from directly executing cloud actions. It emits structured proposals for validation. This separation is necessary because LLMs can hallucinate commands, misinterpret telemetry, or be manipulated through prompt injection [48], [49].

D. Policy, execution, verification, and rollback

The policy and safety engine evaluates action allowlists, RBAC constraints, OPA rules, blast radius, change windows, data sensitivity, rollback availability, and current error-budget state. Kubernetes RBAC, admission control, and audit logs provide native control-plane primitives for limiting and recording actions [26]-[28]. OPA can decouple policy decisions from enforcement [46]. The execution layer applies approved actions through Kubernetes operators, Terraform, Ansible, Argo Workflows, serverless functions, or cloud APIs. The verification layer compares post-action telemetry against SLOs and canary health. If recovery is not observed or collateral damage appears, rollback is executed by default.

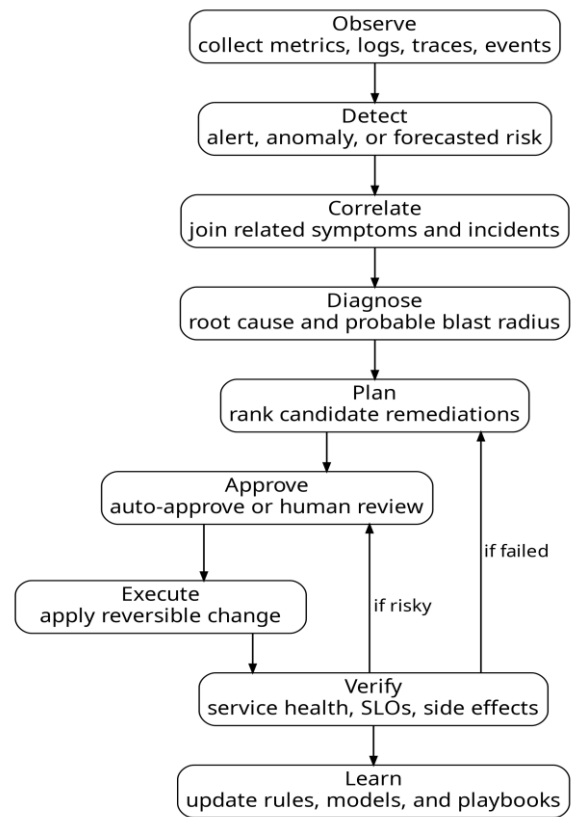


Figure 3. Incident lifecycle from detection to learning.

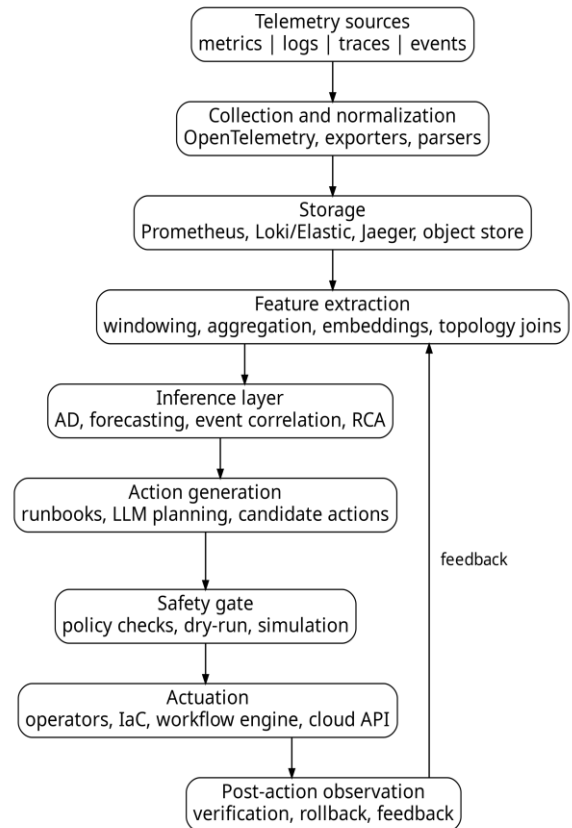


Figure 4. Telemetry-to-remediation pipeline.

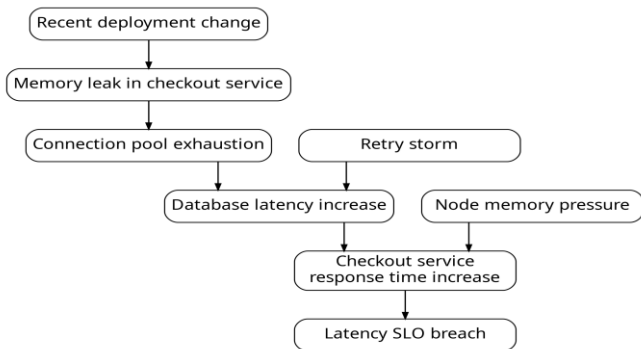


Figure 5. Example causal graph for root cause analysis.

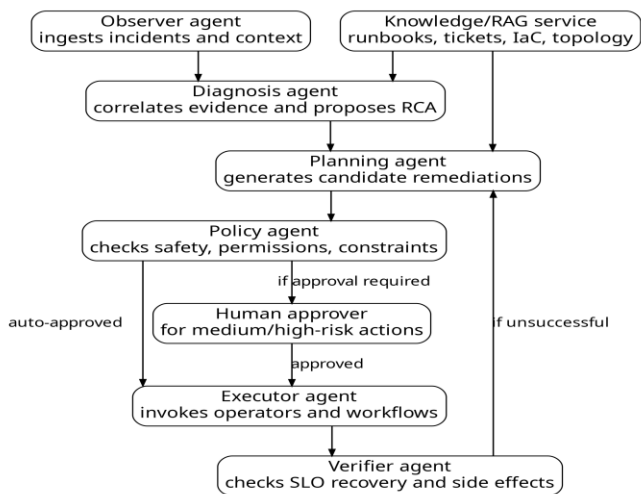


Figure 6. Multi-agent decomposition for agentic cloud operations.

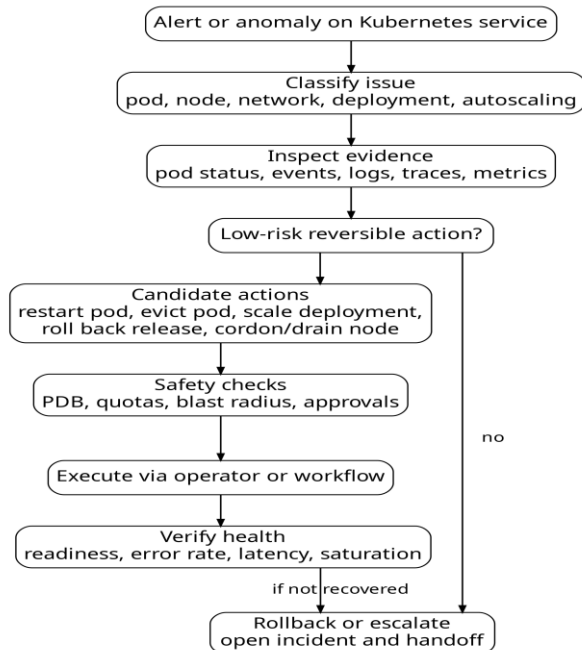


Figure 7. Kubernetes self-healing workflow with safety checks.

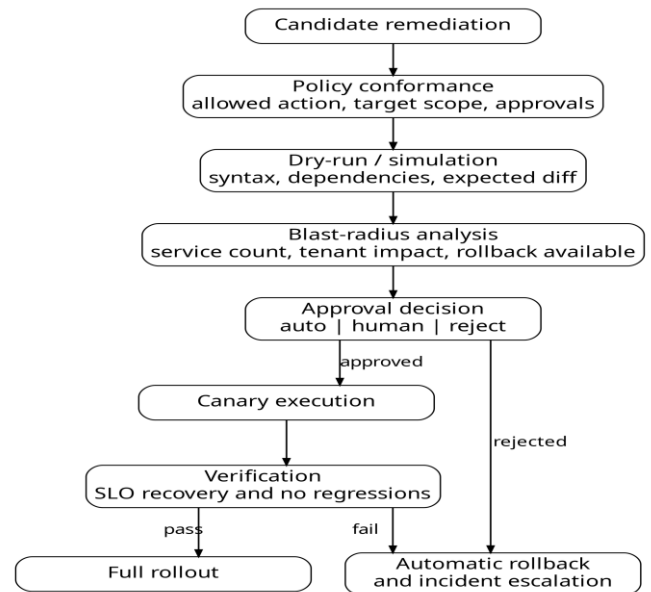


Figure 8. Safety guardrail and rollback workflow.

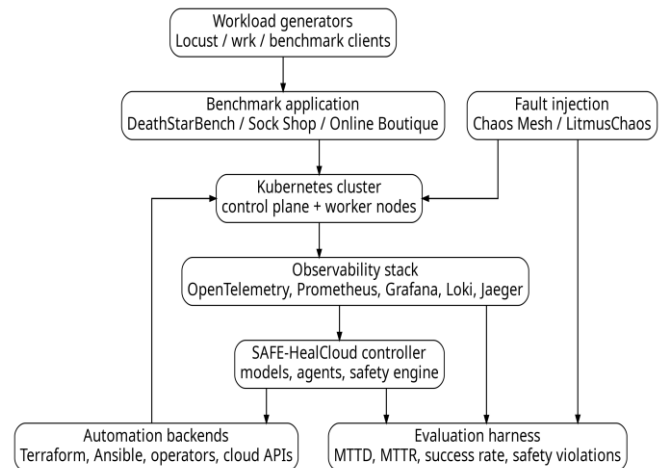


Figure 9. Reproducible evaluation testbed design.

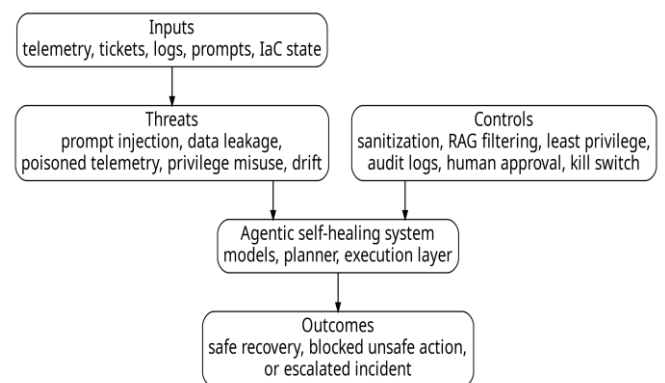


Figure 10. Threat and risk model for agentic self-healing.

FORMAL MODEL AND ALGORITHMS

Let the cloud system state at decision time (t) be

$$[s_t = \langle G_t, X_t, C_t, D_t, B_t \rangle,$$

where (G_t) is the service-dependency graph, (X_t) is workload and resource state, (C_t) is configuration and deployment state, (D_t) is recent incident and change history, and (B_t) is the reliability budget state. The observability vector is

$$[o_t = [m_t, l_t, \tau_t, e_t, c_t],$$

where (m_t) denotes metrics, (l_t) logs, (τ_t) traces, (e_t) events, and (c_t) configuration or topology features. A failure state is represented as

$$[F(s_t)=1 \quad \text{if} \quad \text{SLOViolation}(s_t) \quad \text{or} \quad \text{UnhealthyControlPlane}(s_t) \quad \text{or} \quad \text{UnsafeConfig}(s_t).]$$

The action space includes no-op, restart pod, reschedule workload, scale out, scale up, roll back release, shift traffic, revert IaC, rotate certificate, drain node, throttle workload, open incident, and escalate. A remediation policy is

$$[\pi_\theta(a_t | o_t, K_t),]$$

where (K_t) includes runbooks, prior incidents, policies, dependency graphs, ownership metadata, and cloud-provider constraints. The reward function for constrained optimization is

$$[R_t = \lambda_1 \Delta \text{Availability}_t + \lambda_2 \Delta \text{SLOCompliance}_t - \lambda_3 \text{MTTR}_t - \lambda_4 \text{CostOverhead}_t - \lambda_5 \text{LatencyOverhead}_t - \lambda_6 \text{HumanIntervention}_t - \lambda_7 \text{SafetyViolation}_t.]$$

The controller solves

$$[\max_{\pi} \mathbb{E}[\sum_{t=0}^T \gamma^t R_t] \quad \text{subject to} \quad a_t \in \mathcal{A}_{\text{safe}}(s_t),]$$

where $(\mathcal{A}_{\text{safe}})$ is defined by policy, privilege, blast-radius, compliance, and rollback constraints. Reliability and availability are measured as

$$[\text{Availability} = \frac{\text{successful service time}}{\text{total service time}},]$$

$$[\text{MTTD} = \frac{1}{N} \sum_i (t_i^{\text{detect}} - t_i^{\text{start}}), \\ \text{MTTR} = \frac{1}{N} \sum_i (t_i^{\text{recover}} - t_i^{\text{detect}}).]$$

A. Real-time anomaly detection

Algorithm listing

```

Algorithm 1: STREAM-ANOMALY-DETECT
Input: telemetry window  $W_t$ , baseline models  $B$ , graph  $G_t$ , thresholds  $\tau_c$  and  $\tau_p$ 
Output: clustered incident candidates  $I_t$ 
1: for each service node  $v$  in  $G_t$  do
2:    $x_v \leftarrow$  extract metrics, logs, traces, events, and topology features
3:    $s_{\text{stat}} \leftarrow$  statistical_deviation( $B.\text{stat}[v]$ ,  $x_v$ )
4:    $s_{\text{recon}} \leftarrow$  reconstruction_error( $B.\text{autoencoder}[v]$ ,  $x_v$ )
5:    $s_{\text{trace}} \leftarrow$  trace_path_deviation( $B.\text{trace}[v]$ ,  $x_v$ )
6:    $s_{\text{event}} \leftarrow$  event_deviation( $B.\text{event}[v]$ ,  $x_v$ )
7:    $s_v \leftarrow$  weighted_fusion( $s_{\text{stat}}$ ,  $s_{\text{recon}}$ ,  $s_{\text{trace}}$ ,  $s_{\text{event}}$ )
8:   if  $s_v > \tau_c$  for at least  $\tau_p$  windows then
9:     emit candidate( $v$ ,  $s_v$ , evidence= $x_v$ )
10:  end if
11: end for
12: cluster candidates by topology, time, and trace context
13: return incident candidates  $I_t$ 

```

B. Dependency-graph RCA

Algorithm listing

```

Algorithm 2: GRAPH-RCA
Input: incident  $I$ , graph  $G=(V,E)$ , anomaly scores  $A(v)$ , change events  $C$ 
Output: ranked root causes  $R$ 
1: initialize  $S(v) \leftarrow A(v)$  for each anomalous node  $v$ 
2: for each node  $v$  do
3:   parent  $\leftarrow$  aggregate_parent_scores( $G$ ,  $v$ ,  $A$ )
4:   child  $\leftarrow$  aggregate_child_scores( $G$ ,  $v$ ,  $A$ )
5:   change  $\leftarrow$  recent_change_signal( $C$ ,  $v$ )
6:   causal  $\leftarrow$  test_temporal_and_causal_consistency( $G$ ,  $v$ )
7:    $S(v) \leftarrow \alpha * A(v) + \beta * \text{change} + \gamma * \text{causal}$ 

```

```

+      delta*parent      -      epsilon*child
8:      end      for
9: propagate scores backward over plausible cause
paths
10: prune nodes inconsistent with temporal ordering
11: return top-k nodes and explanations sorted by S(v)

```

C. LLM-assisted remediation planning

Algorithm listing

```

[1] Algorithm 3: RAG-REMEDiate
Input: ranked causes R, incident context Q,
knowledge base K
Output: candidate plans P
1: D <- retrieve runbooks, incidents, policies, IaC, and
docs relevant to R and Q
2: T <- assemble grounded context with evidence
identifiers
3: P_raw <- LLM.generate(structured_plan_schema,
context=T,
constraints={allowed_actions, rollback_required,
evidence_required})
4: parse P_raw into structured actions
5: attach evidence, expected effect, verification query,
and rollback action
6: reject actions lacking executable mapping,
evidence, or rollback
7: return P

```

D. Safety-aware remediation approval

Algorithm listing

```

[2] Algorithm 4: SAFE-APPROVE
Input: plan p, policies Pi, risk model rho,
environment state s
Output: approve, canary, require_human, or reject
1: if not policy_allow(Pi, p, s) then return reject
2: if violates_least_privilege(p) then return reject
3: if missing_rollback(p) then return reject
4: if exceeds_blast_radius(p, s) then return
require_human
5: r <- rho(p, s)

```

```

6: if r < low_risk_threshold then return approve
7: if r < medium_risk_threshold then return canary
8: if r < high_risk_threshold then return
require_human
9: return reject

```

E. Reinforcement-learning-based remediation optimization

Algorithm listing

```

[3] Algorithm 5: SHIELDED-RL-OPTIMIZE
Input: state s_t, safe action set A_safe(s_t), reward R,
policy pi_theta
Output: improved policy pi_theta_star
1: observe s_t
2: sample proposed action a_t ~ pi_theta(. | s_t)
3: if a_t not in A_safe(s_t) then
4:   a_t <- safe_projection(a_t, A_safe(s_t))
5: end if
6: execute a_t in simulator, replay, or shadow mode
7: observe s_{t+1}, reward r_t, and safety
interventions
8: update pi_theta with constrained policy
optimization
9: evaluate against held-out fault scenarios
10: promote only after regression and safety gates pass

```

F. Rollback and verification loop

Algorithm listing

```

[4] Algorithm 6: VERIFY-OR-ROLLBACK
Input: executed action a, horizon H, SLO targets Z,
rollback a_inv
Output: success, rollback, or escalation
1: apply a to canary or bounded scope
2: for t in 1..H do
3:   collect post-action telemetry o_t
4:   if safety_violation(o_t) then execute a_inv;
return rollback
5:   if collateral_regression(o_t) then execute a_inv;
return rollback
6:   if SLO_recovered(o_t, Z) and

```

```

confidence_high(o_t)      then      continue
7:  if confidence_below_threshold(o_t) then execute
a_inv;                      return      rollback
8:                      end          for
9: promote if recovery persists across verification
window
10: return success

```

EXPERIMENTAL DESIGN

A credible evaluation must test the whole self-healing loop, not only one model. The proposed testbed deploys benchmark microservices on Kubernetes with OpenTelemetry, Prometheus, Grafana, Loki or Elasticsearch, Jaeger, and a fault-injection framework such as Chaos Mesh or LitmusChaos. For managed-cloud experiments, AWS Fault Injection Service can complement open-source chaos tooling [39]-[42]. Execution is performed through Kubernetes operators, Argo Workflows, Terraform, Ansible, and scoped cloud APIs.

Candidate workloads include DeathStarBench, Sock Shop, Online Boutique, and TrainTicket. DeathStarBench was introduced as an open-source microservices benchmark suite covering realistic end-to-end services [22]. Sock Shop and Online Boutique are widely used cloud-native demo applications. TrainTicket provides a larger service graph and can stress dependency-aware RCA. For static and

interactive RCA evaluation, OpenRCA, RCAEval, AIOpsLab, and Cloud-OpsBench provide emerging benchmarks for LLM and agentic cloud operations [8], [12], [13], [65].

The experiment has four stages. Stage 1 collects baseline telemetry under normal workloads and deployments. Stage 2 injects controlled failures: pod crash loops, CPU pressure, memory pressure, packet loss, latency injection, node failure, database saturation, misconfigured autoscaling, certificate expiry, failed deployment, and IaC drift. Stage 3 runs SAFE-HealCloud in three modes: assisted operations, semi-autonomous healing, and fully autonomous healing under guardrails. Stage 4 compares outcomes against manual runbooks and rule-based automation.

The evaluation should log all inputs and actions: injected fault specification, workload intensity, telemetry snapshot, RCA ranking, LLM context, retrieved evidence, proposed actions, policy decisions, human approvals, executed commands, verification queries, rollback events, and final outcome. Without these traces, it is impossible to audit whether recovery was caused by the system, chance, or unreported human intervention. Each experiment should also record model version, prompt or agent policy, random seed, Kubernetes version, node size, workload generator version, benchmark commit hash, and cloud provider or local cluster environment.

EVALUATION METRICS

Metric	Definition	Purpose
MTTD	Time from fault onset to reliable detection	Measures detection responsiveness
MTTR	Time from detection to verified recovery	Measures operational recovery value
Failure prediction accuracy	Correctness of pre-incident warnings	Measures proactive capability
Precision, recall, F1	Standard detection or classification quality	Balances false alarms and misses
False positive rate	Healthy states incorrectly flagged	Measures alert fatigue risk

Metric	Definition	Purpose
False negative rate	Incidents missed	Measures reliability exposure
RCA top-k accuracy	True root cause in top-k candidates	Measures diagnostic usefulness
Remediation success rate	Actions restoring target health	Measures practical healing
Rollback success rate	Rollbacks restoring stable state	Measures safety net quality
SLO violation reduction	Change in time outside SLO	Measures user-facing reliability
Error-budget preservation	Reduced budget burn per window	Aligns with SRE governance
Cost overhead	Extra compute, storage, and API cost	Prevents inefficient healing
Latency overhead	Instrumentation and control latency	Captures runtime burden
Human intervention reduction	Reduced manual steps or pages	Measures toil reduction
Safety violation rate	Policy violations or collateral damage	Measures governance quality
Explainability score	Operator-rated diagnostic clarity	Measures trust and adoption

The most important distinction is between **model metrics** and **operational metrics**. A detector can have high F1 but still be operationally weak if it fires too late. An RCA model can rank the correct component but fail to propose a safe action. A remediation can restore a pod but violate a service SLO. Self-healing must therefore be evaluated through recovery outcomes and safety constraints.

FINDINGS

A. Finding 1: Multimodal observability is a prerequisite

The literature consistently shows that logs, metrics, traces, events, and topology capture different aspects of failures [4], [5], [17]. Metrics expose quantitative deviation, traces reveal propagation paths, logs contain semantic local evidence, events capture deployments or control-plane actions, and topology encodes dependencies. A system that observes only one signal will misdiagnose many incidents. The practical implication is that organizations should improve telemetry correlation before deploying advanced autonomy. The limitation is cost and

instrumentation burden. Future work should define standardized multimodal cloud incident schemas and benchmark datasets.

B. Finding 2: Dependency-aware and causal methods improve diagnosis

Causal and graph-based RCA methods such as CIRCA, Chain-of-Event, and GAMMA explicitly model propagation [15]-[18]. This matters because the most visible symptom is often downstream. Dependency-aware methods can rank upstream causes and provide explainable paths. The practical implication is that service maps, ownership graphs, and change graphs should be treated as reliability assets. The limitation is graph staleness. Future work should combine online graph reconstruction with uncertainty-aware causal inference.

C. Finding 3: Learning-based control is mature for resource adaptation, less mature for open-ended remediation

FIRM and related work show that learning-based resource management can reduce SLO violations in microservices [19]. RL surveys indicate strong interest in cloud scheduling and resource optimization [20],

[21]. However, the action space for general incident remediation is much broader and riskier. The practical implication is to deploy learning first in constrained domains such as autoscaling, contention mitigation, queue control, and canary promotion. The limitation is transferability from simulation to production. Future work should develop high-fidelity cloud incident simulators and safe offline RL benchmarks.

D. Finding 4: LLMs are useful copilots but unreliable autonomous operators

LLM-based RCA and incident systems can retrieve evidence, summarize logs, draft hypotheses, and recommend mitigation [9]-[11]. Yet OpenRCA and process-level analyses show that LLM agents often fail on realistic RCA tasks [12], [14]. The practical implication is that LLMs should be used for triage, explanation, runbook retrieval, and candidate plan drafting, while execution remains policy-gated. The limitation is that current benchmarks remain fragmented and may not cover all production failure modes. Future work should evaluate tool-use behavior, evidence grounding, and safety, not only final answer accuracy.

E. Finding 5: Safety architecture matters as much as model quality

OWASP identifies prompt injection and excessive agency as central LLM risks [48], [49], while NIST AI RMF emphasizes accountability, validity, reliability, safety, security, resilience, explainability, privacy, and fairness [50]. In cloud operations, the consequence of unsafe autonomy can be service outage, data exposure, or destructive infrastructure change. The practical implication is to separate reasoning from execution

and enforce least privilege, policy-as-code, audit logs, approval gates, and rollback. The limitation is organizational overhead. Future work should standardize safety metrics for self-healing systems.

F. Finding 6: Interactive benchmarks are replacing static datasets

AIOpsLab, OpenRCA, RCAEval, and Cloud-OpsBench reflect a shift toward interactive, reproducible evaluation [8], [12], [13], [65]. This is necessary because agentic systems must query tools, retrieve evidence, and decide when to stop. The practical implication is that organizations should test agents in controlled environments before production use. The limitation is that benchmark realism and reproducibility are difficult to balance. Future work should create shared benchmark standards, comparable leaderboards, and trace-level evaluation protocols.

G. Finding 7: Self-healing IaC is promising but immature

Infrastructure-as-code repair using constrained LLM technologies demonstrates a path toward automatic detection and correction of IaC errors [59]. Terraform documentation also warns that drift between state and real infrastructure can lead to unintended changes [47]. Runtime self-healing must therefore reconcile short-term repair with long-term desired state. The practical implication is to begin with AI-generated IaC pull requests and policy-checked recommendations rather than fully autonomous applies. The limitation is sparse evaluation data. Future work should combine LLM repair, static analysis, policy checking, and formal verification.

COMPARATIVE TABLES

A. Rule-based, ML-based, and LLM-agent-based healing

Dimension	Rule-based	ML-based	LLM-agent-based
Best fit	Stable, known, reversible incidents	Recurrent patterns in telemetry	Heterogeneous, long-context reasoning

Dimension	Rule-based	ML-based	LLM-agent-based
Strength	Deterministic and explainable	Adaptive detection and ranking	Evidence synthesis and plan drafting
Weakness	Brittle outside encoded cases	Drift and label scarcity	Hallucination and unsafe agency
Safety	High for small blast radius	Depends on validation	Requires strict guardrails
Maturity	Production proven	Mature for detection, mixed for remediation	Promising but early

B. AI techniques and best use cases

Technique	Best use case	Strength	Weakness
Thresholds	Simple probe failures	Transparent	Brittle
Unsupervised anomaly detection	Rare failure detection	Label efficient	Weak explanation
Forecasting	Capacity and SLO prediction	Lead time	Regime shift
Supervised classification	Failure triage	Accurate with labels	Label cost
Causal inference	RCA under propagation	Cause-effect reasoning	Assumption sensitive
GNNs	Topology-aware localization	Dependency modeling	Graph quality
RL	Resource adaptation	Sequential optimization	Unsafe exploration
LLM/RAG	Incident synthesis	Long-context reasoning	Hallucination
Multi-agent systems	Modular operations tasks	Specialization	Coordination failure

C. Failure types and recommended remediation strategies

Failure type	Detection signals	Candidate remediation	Risk tier
Pod crash loop	Restart count, logs, readiness	Restart, isolate, rollback deployment	Low-medium
Memory leak	Rising RSS, OOM kills	Scale out, restart instance, capture heap	Low-medium
Latency spike	p95/p99, trace elongation	Scale out, shift traffic, rollback	Medium
DB pool exhaustion	Timeouts, queueing, pool metrics	Reduce concurrency, tune pool, restart caller	Medium
Node failure	Node NotReady, evictions	Cordon, drain, reschedule	Low

Failure type	Detection signals	Candidate remediation	Risk tier
Autoscaling misconfiguration	Replica oscillation	Freeze policy, restore known-good HPA	Medium
Expired TLS certificate	Handshake failures, cert age	Renew or rotate certificate	Low
Cost anomaly	Billing spike, scale increase	Cap scale, throttle job, alert owner	Medium
Network partition	Packet loss, trace gaps	Failover or route change	High
Failed deployment	Readiness failure, error spike	Rollback release	Medium
IaC misconfiguration	Drift, failed plan	Open PR or approved revert	High

D. Existing tools and frameworks

Tool or framework	Role	Relevance
Kubernetes controllers/operators	Desired-state actuation	Native execution substrate
OpenTelemetry	Telemetry generation/export	Cross-signal evidence
Prometheus	Metrics	Detection and SLO tracking
Loki/Elasticsearch	Logs	Semantic RCA evidence
Jaeger	Traces	Propagation and dependency analysis
OPA	Policy-as-code	Safety broker
Terraform	Declarative infrastructure	IaC reconciliation
Ansible	Procedural automation	Idempotent playbooks
Argo Workflows	Workflow execution	Multi-step remediation
Chaos Mesh/LitmusChaos/AWS FIS	Fault injection	Resilience testing
AIOpsLab/OpenRCA/RCAEval/Cloud-OpsBench	Benchmarks	Reproducible evaluation

E. Public benchmarks and datasets

Benchmark	Scope	Strength
DeathStarBench	Microservice benchmark suite	Realistic end-to-end interactions
Sock Shop	Cloud-native demo application	Easy deployment and testing
Online Boutique	E-commerce microservices	Heterogeneous services
TrainTicket	Large benchmark microservice system	Complex dependency graph

Benchmark	Scope	Strength
OpenRCA	LLM RCA benchmark	Long-context incident evidence
RCAEval	Microservice RCA benchmark	Baselines and failure cases
AIOpsLab	AgentOps environment	Interactive incident lifecycle tasks
Cloud-OpsBench	Agentic RCA benchmark	Deterministic cloud-state cases

F. Risks and mitigations

Risk	Mechanism	Mitigation
Unsafe action	Correct diagnosis, harmful remediation	Canarying, approval, rollback
Hallucinated plan	Fabricated command or resource	RAG grounding, schemas, validators
Privilege escalation	Excessive cloud credentials	Least privilege, scoped broker
Data leakage	Sensitive logs in retrieval	Redaction and access control
Prompt injection	Malicious log text	Instruction/data separation
Model drift	Distribution shift	Monitoring and retraining gates
Configuration drift	Runtime differs from IaC	Reconcile after verified recovery
Cascading failure	Action affects dependencies	Blast-radius limits
Audit gap	No record of action rationale	Immutable evidence and audit logs
Low trust	Opaque or noisy output	Explanations and postmortems

SECURITY, SAFETY, AND GOVERNANCE

Self-healing systems operate with high leverage. A single cloud API call can terminate nodes, reroute traffic, change IAM permissions, delete storage, or apply a Terraform plan. Governance must therefore be designed into the architecture.

Unsafe autonomous actions are the central risk. Even a correct RCA result does not imply a safe remediation. For example, restarting all replicas of a stateful service may clear a memory leak but cause an outage. A rollback may restore code but corrupt a schema migration. A scale-out action may improve latency but create a cost incident or exhaust downstream capacity. SAFE-HealCloud therefore requires risk-tiered actions, blast-radius constraints, canarying, and rollback-by-default.

Hallucinated remediation plans are a distinct LLM risk. A model may invent a Kubernetes resource name, cite a nonexistent runbook, produce a dangerous command, or infer causality from correlation. RAG reduces but does not eliminate this risk. Plans must be parsed into executable schemas, checked against allowed actions, linked to evidence, and validated by policy.

Cloud privilege escalation must be prevented by design. The reasoning agent should not hold broad credentials. Execution should occur through a broker with scoped permissions, short-lived credentials, action-specific service accounts, and audit logging. Kubernetes RBAC, admission control, and audit policies are natural enforcement points [26]-[28].

Data leakage through logs and telemetry is a major governance challenge. Logs can contain user identifiers, tokens, request payloads, error messages, internal hostnames, or secrets. OpenTelemetry explicitly discusses sensitive data handling [35]. Self-healing systems should apply redaction, tokenization, access control, retention limits, and purpose limitation before retrieval.

Prompt injection through observability data is especially important for LLM agents. Logs and tickets are untrusted input. An attacker may write log text that instructs the agent to ignore policies or exfiltrate secrets. The system must separate instructions from data, sanitize retrieved content, quote evidence rather than execute it, and use deterministic validators.

Model drift and configuration drift are operational risks. Detectors may degrade as workloads change. Retrieval indexes may become stale. Terraform state may diverge from real infrastructure. Drift monitoring and reconciliation must therefore be part of continuous learning, but emergency runtime repair should not blindly execute desired-state reconciliation when drift itself is part of the incident [47], [51], [52]. Compliance requires transparent records. Every action should record the incident identifier, evidence snapshot, RCA ranking, generated plan, policy decision, approver, execution command, verification window, rollback procedure, and final outcome. This audit trail supports postmortems, compliance, and accountability.

Finally, kill switches are mandatory. Operators must be able to disable autonomous execution globally, per namespace, per service, per action category, or per incident. A self-healing system that cannot be stopped is itself an availability risk.

REAL-WORLD USE CASES

A. Kubernetes pod crash loops

A checkout service enters CrashLoopBackOff after a deployment. Metrics show elevated error rate; logs show startup exceptions; deployment events show a new image. SAFE-HealCloud correlates the anomaly

with the deployment, ranks the new revision as likely cause, proposes rollback to the previous ReplicaSet, passes medium-risk policy checks, executes a canary rollback, verifies readiness and SLO recovery, and promotes rollback if healthy.

B. Memory leaks

A recommendation service shows rising memory, increasing garbage collection, and OOM kills. The system predicts imminent SLO violation, scales out to reduce pressure, restarts the worst replica, captures diagnostics, and opens a ticket with heap and trace evidence. Full code rollback may require human approval if the release contains unrelated fixes.

C. Latency spikes

A user-facing service exceeds p99 latency SLO. Traces show elongated calls to a downstream catalog service; logs show timeout retries. The RCA engine identifies the catalog service as a bottleneck and recent CPU saturation on its node. The planner proposes traffic shifting and replica scaling. Verification checks SLO recovery and downstream database saturation.

D. Database connection pool exhaustion

Multiple services report database timeouts. Flat anomaly ranking highlights front-end errors, but graph RCA identifies database pool exhaustion in a shared dependency. The system reduces caller concurrency, increases pool capacity within a safe limit, and verifies that database CPU and lock waits do not worsen.

E. Node failures

A node becomes NotReady and pods are evicted. Kubernetes reschedules replicas, but SAFE-HealCloud validates service-level recovery, checks pod disruption budgets, records the node failure, and opens a hardware or cloud-provider incident if replacement capacity is delayed.

F. Misconfigured autoscaling

An HPA policy oscillates replicas due to an incorrect target metric. The system detects replica thrash, elevated cost, and unstable latency. It proposes restoring the previous HPA policy from version

control, freezes autoscaling temporarily, and requires approval if the change affects a critical service.

G. Expired TLS certificates

Handshake failures appear at an ingress gateway. Certificate age telemetry shows expiration. The planner proposes renewal through the managed issuer, verifies the new certificate chain, and checks external endpoint health. Because the action is narrow and reversible, it can be low risk.

H. Cloud cost anomalies

Billing telemetry shows an unexpected spike after a batch job scales out. The system correlates cost anomaly with deployment and HPA events. It proposes a temporary scaling cap and owner notification. Because cost actions can affect availability, the policy engine balances cost reduction against SLO risk.

I. Network partition

A regional network partition causes packet loss and trace gaps. The system detects correlated failures across services and zones. It recommends traffic failover, but because cross-region routing is high impact, human approval is mandatory. Verification checks latency, error rate, capacity, and data consistency.

J. Failed deployment

A canary rollout fails readiness checks and increases errors. The system stops rollout, reverts to the previous version, verifies SLO recovery, and attaches telemetry to the deployment record. This is a mature case for semi-autonomous healing because progressive delivery already provides bounded rollback mechanisms.

K. Terraform/IaC misconfiguration

A Terraform change modifies a security group or autoscaling policy incorrectly. The system detects drift and failed health checks, retrieves the plan, identifies the risky change, and opens a revert pull request. Direct autonomous terraform apply is rejected unless the action is pre-approved, narrow, and fully reversible.

DISCUSSION

The evidence supports a sober conclusion: AI can improve cloud reliability today, but unrestricted autonomous cloud operations remain premature. The appropriate adoption model is graduated autonomy.

Assisted operations are ready now. AI can summarize incidents, retrieve related failures, identify recent changes, draft timelines, suggest candidate causes, generate postmortem outlines, and recommend runbook steps. Human engineers remain responsible for action.

Semi-autonomous healing is practical for bounded actions. Restarting pods, rolling back failed canaries, scaling replicas within limits, rotating certificates, cordoning nodes, or freezing misconfigured autoscalers can be automated if the system has evidence, policy approval, scoped credentials, verification, and rollback.

Fully autonomous healing should be reserved for narrow domains with strong invariants. Multi-region failover, database-changing actions, IAM modification, broad Terraform applies, and cross-service traffic engineering should remain human-approved. These actions are too high impact for current LLM-agent reliability and benchmark maturity.

The central bottleneck is not only model quality. It is the quality of observability, the accuracy of dependency graphs, the availability of safe execution surfaces, and the maturity of evaluation. A powerful model operating on poor telemetry will fail. A correct diagnosis without guardrails may be unsafe. A successful demo without benchmark evidence may not generalize. Therefore, organizations should treat self-healing as an engineering system that combines SRE, platform engineering, security, MLOps, and governance.

LIMITATIONS

This paper has five limitations. First, it is a structured synthesis and architecture proposal, not a report of new production experiments. Second, the field is changing rapidly, especially LLM-agent benchmarks,

and some relevant work is available only as preprints. Third, large cloud providers possess internal incident datasets that are not publicly reproducible, which limits independent validation. Fourth, benchmark realism remains incomplete: no public benchmark captures all operational, organizational, security, and compliance constraints of production cloud platforms. Fifth, the framework assumes mature observability and policy infrastructure that many organizations do not yet possess.

These limitations do not invalidate the framework; they define the next empirical steps. SAFE-HealCloud should be implemented in an open testbed, evaluated across multiple benchmarks and fault types, and compared against rule-based automation, human runbooks, and LLM-only agents.

REPRODUCIBILITY GUIDANCE

A reproducible study should publish: cluster configuration, workload generator, benchmark application version, fault injection manifests, telemetry schemas, feature extraction code, RCA implementation, retrieval corpus, prompts or agent policies, policy rules, remediation playbooks, execution traces, verification queries, rollback criteria, and evaluation scripts. Each experiment should record random seeds, workload intensity, fault start and end times, resource quotas, cloud region or local cluster profile, model versions, and human approval events.

For LLM-based components, reproducibility requires more than final text output. Authors should log retrieved documents, tool calls, intermediate hypotheses, structured plans, validator outputs, and policy decisions. This is necessary because agentic failures often arise from process errors such as premature stopping, wrong tool selection, stale retrieval, or ungrounded reasoning.

FUTURE RESEARCH AGENDA

Research direction	Open problem
Agentic AIOps benchmarks	Evaluate tool use, process quality, and recovery outcomes
Standardized datasets	Create shared multimodal telemetry schemas
Explainable RCA	Provide faithful causal paths, not only rankings
Safe LLM agents	Combine RAG, policy, schemas, and verifiers
Self-healing IaC	Repair code while preserving desired-state safety
Multi-cloud healing	Coordinate across providers and control planes
Edge-cloud healing	Operate under intermittent connectivity and constraints
Green cloud recovery	Optimize energy and carbon alongside reliability
Formal verification	Prove safety of remediation actions where possible
Human-AI collaboration	Design trust, feedback, escalation, and accountability workflows

CONCLUSION

AI-driven self-healing for cloud infrastructure is best understood as a disciplined reliability architecture, not as a single model or autonomous agent. Modern cloud failures are complex because they arise from interactions among services, configurations, deployments, platforms, networks, and humans. Reactive monitoring and manual incident response will remain important, but they are insufficient for the scale and speed of cloud-native operations.

This paper synthesized the literature on AIOps, AgentOps, RCA, anomaly detection, graph learning, reinforcement learning, LLM-based operations, Kubernetes self-healing, observability, chaos

engineering, and self-healing IaC. It proposed SAFE-HealCloud, a safety-aware framework that integrates telemetry, detection, prediction, causal diagnosis, LLM/RAG planning, policy guardrails, controlled execution, verification, rollback, human approval, and continuous learning. It also defined formal models, algorithms, evaluation metrics, reproducibility guidance, use cases, and governance controls.

The most important practical recommendation is graduated autonomy. Low-risk reversible remediation can already be automated. Medium-risk changes should be canaried and policy-gated. High-risk actions should remain human-approved until benchmarks, explainability, and formal validation improve. In this model, AI augments SRE and platform engineering judgment while reducing toil and accelerating recovery.

If designed and governed correctly, AI-driven self-healing can improve digital resilience far beyond cloud platforms. Faster and safer recovery can strengthen healthcare systems, financial services, education platforms, public-sector infrastructure, climate and scientific computing, and global digital services. The goal should not be autonomy at any cost; it should be trustworthy autonomous recovery in service of reliable, accountable, and resilient digital society.

DECLARATIONS

A. Ethics approval and consent to participate

Not applicable. This study does not involve human participants, human data, animals, or clinical interventions.

B. Consent for publication

Not applicable.

C. Availability of data and materials

No new empirical dataset was generated in this conceptual review and framework paper. Reproducibility guidance, experimental specifications, sample Kubernetes YAML, OpenTelemetry pipeline configuration, and remediation playbook examples are provided in the Appendix. If an implementation is

developed, the authors should deposit source code, manifests, benchmark versions, anonymized telemetry, and evaluation scripts in a public repository and cite the repository DOI.

D. Competing interests

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper. [Revise if applicable.]

E. Funding

This research received no specific grant from any funding agency in the public, commercial, or not-for-profit sectors. [Revise if applicable.]

F. CRediT author statement

[Author 1]: Conceptualization, Methodology, Investigation, Formal analysis, Writing - original draft, Writing - review and editing. [Author 2]: Methodology, Validation, Software, Visualization, Writing - review and editing. [Author 3]: Supervision, Project administration, Resources, Writing - review and editing. [Revise to reflect actual contributions.]

G. Acknowledgements

The authors thank [names or groups] for feedback on cloud reliability, SRE practice, and AIOps evaluation. [Optional.]

H. Declaration of generative AI and AI-assisted technologies in the manuscript preparation process

During the preparation of this work the author(s) used OpenAI ChatGPT to assist with drafting, organization, literature synthesis, and language editing. After using this tool, the author(s) reviewed and edited the content as needed and take(s) full responsibility for the content of the published article, including source verification, accuracy, originality, and compliance with journal policies.

References

- [1]. J. O. Kephart and D. M. Chess, "The vision of autonomic computing," *Computer*, vol. 36, no.

- 1, pp. 41-50, 2003. doi:10.1109/MC.2003.1160055.
- [2]. S. R. White, J. E. Hanson, I. Whalley, D. M. Chess, and J. O. Kephart, "An architectural approach to autonomic computing," IBM Research, 2004.
- [3]. A. Bucchiarone, C. Guidi, I. Lanese, N. Bencomo, and J. Spillner, "A MAPE-K approach to autonomic microservices," in Proc. IEEE International Conference on Software Architecture Companion (ICSA-C), 2022.
- [4]. J. Soldani and A. Brogi, "Anomaly detection and failure root cause analysis in (micro)service-based cloud applications: A survey," ACM Computing Surveys, vol. 55, no. 3, 2022. doi:10.1145/3501297.
- [5]. S. Zhang et al., "Failure diagnosis in microservice systems: A comprehensive survey and analysis," 2025. [Preprint/manuscript].
- [6]. L. Zhang et al., "A survey of AIOps for failure management in the era of large language models," arXiv:2406.11213, 2024. [Preprint].
- [7]. M. Shetty et al., "Building AI agents for autonomous clouds: Challenges and design principles," arXiv:2407.12165, 2024. [Preprint].
- [8]. Y. Chen et al., "AIOpsLab: A holistic framework to evaluate AI agents for enabling autonomous clouds," arXiv:2501.06706, 2025. [Preprint].
- [9]. T. Ahmed et al., "Recommending root-cause and mitigation steps for cloud incidents using large language models," in Proc. IEEE/ACM International Conference on Software Engineering (ICSE), 2023.
- [10]. Y. Chen et al., "Automatic root cause analysis via large language models for cloud incidents," arXiv:2305.15778, 2023. [Preprint].
- [11]. D. Roy et al., "Exploring LLM-based agents for root cause analysis," in Proc. ACM International Conference on the Foundations of Software Engineering (FSE), Industry Track, 2024.
- [12]. J. Xu et al., "OpenRCA: Can large language models locate the root cause of software failures?" International Conference on Learning Representations (ICLR), 2025.
- [13]. L. Pham, H. Zhang, H. Ha, F. Salim, and X. Zhang, "RCAEval: A benchmark for root cause analysis of microservice systems with telemetry data," arXiv:2412.17015, 2024. [Preprint].
- [14]. T. Kim et al., "Why do AI agents systematically fail at cloud root cause analysis?" arXiv:2602.09937, 2026. [Preprint].
- [15]. M. Li et al., "Causal inference-based root cause analysis for online service systems with intervention recognition," in Proc. ACM SIGKDD Conference on Knowledge Discovery and Data Mining, 2022.
- [16]. Z. Yao et al., "Chain-of-Event: Interpretable root cause analysis for microservices through automatically learning weighted event causal graph," in Proc. ACM FSE Companion, 2024.
- [17]. S. Xie et al., "TVDiag: A task-oriented and view-invariant failure diagnosis framework with multimodal data," arXiv:2407.19711, 2024. [Preprint].
- [18]. G. Somashekar et al., "GAMMA: Graph neural network-based multi-bottleneck localization for microservices applications," in Proc. ACM Web Conference (WWW), 2024.
- [19]. H. Qiu et al., "FIRM: An intelligent fine-grained resource management framework for SLO-oriented microservices," in Proc. USENIX OSDI, 2020.
- [20]. Y. Gu et al., "Deep reinforcement learning for job scheduling and resource management in cloud computing: An algorithm-level review," arXiv:2501.01007, 2025. [Preprint].
- [21]. G. Zhou et al., "Deep reinforcement learning-based methods for resource scheduling in cloud computing: A review," 2024.
- [22]. Y. Gan et al., "An open-source benchmark suite for microservices and their hardware-software implications for cloud and edge systems," in Proc. ASPLOS, 2019.

- [23]. Kubernetes Project, "Kubernetes self-healing," documentation, 2025.
- [24]. Kubernetes Project, "Deployments," documentation, 2026.
- [25]. Kubernetes Project, "Operator pattern," documentation, 2025.
- [26]. Kubernetes Project, "Admission control in Kubernetes," documentation, 2026.
- [27]. Kubernetes Project, "Using RBAC authorization," documentation, 2026.
- [28]. Kubernetes Project, "Auditing," documentation, 2025.
- [29]. Kubernetes Project, "Liveness, readiness, and startup probes," documentation, 2026.
- [30]. Kubernetes Project, "Horizontal pod autoscaling," documentation, 2026.
- [31]. Kubernetes Project, "Pod lifecycle," documentation, 2026.
- [32]. OpenTelemetry Project, "OpenTelemetry documentation," 2026.
- [33]. OpenTelemetry Project, "OpenTelemetry logging specification," 2026.
- [34]. OpenTelemetry Project, "Observability primer," 2026.
- [35]. OpenTelemetry Project, "Handling sensitive data," 2026.
- [36]. Prometheus Authors, "Prometheus overview," documentation.
- [37]. Grafana Labs, "Loki overview," documentation.
- [38]. Jaeger Authors, "Jaeger project documentation," documentation.
- [39]. Principles of Chaos Engineering, "Principles of chaos engineering," 2019.
- [40]. Chaos Mesh Authors, "Chaos Mesh overview," documentation.
- [41]. LitmusChaos Authors, "What is Litmus?" documentation.
- [42]. Amazon Web Services, "What is AWS Fault Injection Service?" documentation.
- [43]. Google SRE Book, "Service level objectives," documentation.
- [44]. Google SRE Workbook, "Example error budget policy," documentation.
- [45]. Microsoft Learn, "How to read a service-level agreement (SLA)," documentation, 2026.
- [46]. Open Policy Agent Authors, "Open Policy Agent documentation," documentation.
- [47]. HashiCorp, "Manage resource drift," Terraform documentation.
- [48]. OWASP, "LLM prompt injection prevention cheat sheet," 2025.
- [49]. OWASP GenAI Security Project, "LLM06:2025 excessive agency," 2025.
- [50]. National Institute of Standards and Technology, Artificial Intelligence Risk Management Framework (AI RMF 1.0), NIST AI 100-1, 2023.
- [51]. Google Cloud, "Introduction to Vertex AI Model Monitoring," documentation, 2026.
- [52]. Amazon Web Services, "Data and model quality monitoring with Amazon SageMaker Model Monitor," documentation.
- [53]. Z. Zamanzadeh Darban et al., "Deep learning for time series anomaly detection: A survey," ACM Computing Surveys, 2024.
- [54]. Z. Zhong et al., "A survey of time series anomaly detection methods in the AIOps domain," arXiv:2308.00393, 2023. [Preprint].
- [55]. X. Li et al., "A survey on LLM-based multi-agent systems: Workflow, infrastructure, and challenges," 2024.
- [56]. T. Guo et al., "Large language model based multi-agents: A survey of progress and challenges," 2024.
- [57]. W. Fan et al., "A survey on RAG meeting LLMs: Towards retrieval-augmented large language models," 2024.
- [58]. B. Peng et al., "Graph retrieval-augmented generation: A survey," 2024.
- [59]. J. Diaz-de-Arcaya et al., "Towards the self-healing of infrastructure as code projects using constrained LLM technologies," in Proc. ACM/IEEE International Workshop on

Automated Program Repair (APR), 2024.
doi:10.1145/3643788.3648014.

- [60]. AWS Cost Management, "Detecting unusual spend with AWS Cost Anomaly Detection," documentation.
- [61]. Microsoft Learn, "Identify anomalies and unexpected changes in cost," documentation, 2025.
- [62]. Google Cloud Billing, "View and manage cost anomalies," documentation.
- [63]. M. J. Page et al., "The PRISMA 2020 statement: An updated guideline for reporting systematic reviews," BMJ, vol. 372, article n71, 2021. doi:10.1136/bmj.n71.
- [64]. PRISMA Statement, "PRISMA 2020 flow diagram," documentation.
- [65]. Cloud-OpsBench Authors, "Cloud-OpsBench: A reproducible benchmark for agentic root cause analysis in cloud systems," arXiv:2603.00468, 2026. [Preprint].